

Ciberseguridad

Informe taller IA

Segundo Corte

Yslen Natalia Moreno Gutierrez
maria Belen Peña
Laura Sophia Hernandez

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 22 de octubre de 2026

1. IA local frente a IA en la nube

Tabla 1: *Comparación entre IA local e IA en la nube.*

Criterio	IA local	IA en la nube
Privacidad	Los datos se quedan en el equipo; nadie más los ve.	Los datos viajan al proveedor y quedan sujetos a sus políticas de uso y retención.
Seguridad	La superficie de ataque es el propio equipo, que uno mismo protege.	Riesgo de filtraciones del proveedor, interceptación o acceso de terceros.
Control	Se elige el modelo, la versión y la configuración; funciona sin Internet.	El proveedor puede cambiar el modelo, los precios o las condiciones en cualquier momento.
Cumplimiento legal	Más fácil cumplir la Ley 1581 de 2012 (protección de datos en Colombia).	Implica transferir datos a terceros, muchas veces fuera del país.
Capacidad	Limitada por el hardware: modelos pequeños y más lentos.	Modelos mucho más grandes, rápidos y precisos.
Costo	Solo el hardware; no hay pago por uso.	Suscripción o pago por cada consulta.

Para este taller la diferencia importa mucho: las capturas de tráfico contienen direcciones IP, puertos, nombres de equipos (en la captura VPN aparece el nombre del PC anfitrión, DESKTOP-5LMOD0P) y datos de la infraestructura de red. Subirlas a una IA en la nube sería entregarle esa información a un tercero, que es justo lo que un analista de seguridad debe evitar. Por eso se trabajó con IA local.

2. Modelos de IA local: parámetros y hardware

Dos conceptos ayudan a entender qué modelo puede correr un equipo:

- **Parámetros:** son los "pesos" que el modelo aprendió, y se cuentan en miles de millones (B, del inglés *billion*). Un modelo de 3B tiene 3.000 millones de parámetros. Más parámetros suele significar mejores respuestas, pero también más memoria y más lentitud.
- **Cuantización:** es comprimir esos pesos (por ejemplo de 16 bits a 4 bits, formato Q4_K_M) para que el modelo quepa en un computador normal perdiendo poca calidad. Como regla práctica, en Q4 un modelo ocupa unos 0,6 GB por cada mil millones de parámetros, más 1 o 2 GB para el contexto.

Para ejecutar los modelos se usó **Ollama**, una herramienta gratuita que descarga los modelos ya

cuantizados y los corre con un solo comando. Su catálogo (ollama.com/library) incluye, entre otros, los siguientes:

Tabla 2: *Modelos de IA local disponibles en Ollama (tamaños aproximados en Q4).*

Modelo	Empresa	Parámetros	Tamaño	Hardware recomendado
Llama 3.2	Meta	1B / 3B	1,3 / 2,0 GB	8 GB de RAM, sin GPU
Phi-4 mini	Microsoft	3,8B	2,5 GB	8 GB de RAM
Qwen 3	Alibaba	4B / 8B / 14B	2,5 / 5,2 / 9,3 GB	8 GB RAM (4B); 16 GB RAM u 8 GB VRAM (8B); GPU de 12–16 GB (14B)
Gemma 3	Google	4B / 12B / 27B	3,3 / 8,1 / 17 GB	8 GB RAM (4B) hasta GPU de 24 GB (27B)
DeepSeek-R1 (destilado)	DeepSeek	8B / 14B	5,2 / 9 GB	16 GB de RAM o GPU de 8–12 GB
gpt-oss	OpenAI	20B	14 GB	16 GB de RAM o VRAM
Llama 3.3	Meta	70B	43 GB	Estación de trabajo con varias GPU

Si el modelo cabe en la memoria de la tarjeta gráfica (VRAM) corre en la GPU y responde rápido; si no, Ollama usa la RAM y el procesador, y funciona pero mucho más lento.

3. Análisis del equipo

El computador físico es un portátil con procesador Intel Core i7-1255U (12.ª generación) y 16 GB de RAM. La IA se ejecutó dentro de la máquina virtual **kali clon**, a la que se le asignaron 8 GB de RAM (la mitad de la memoria del equipo) y 4 núcleos. Los datos se tomaron con los comandos `free -h`, `lscpu`, `lspci | grep -i vga` y `df -h /`.

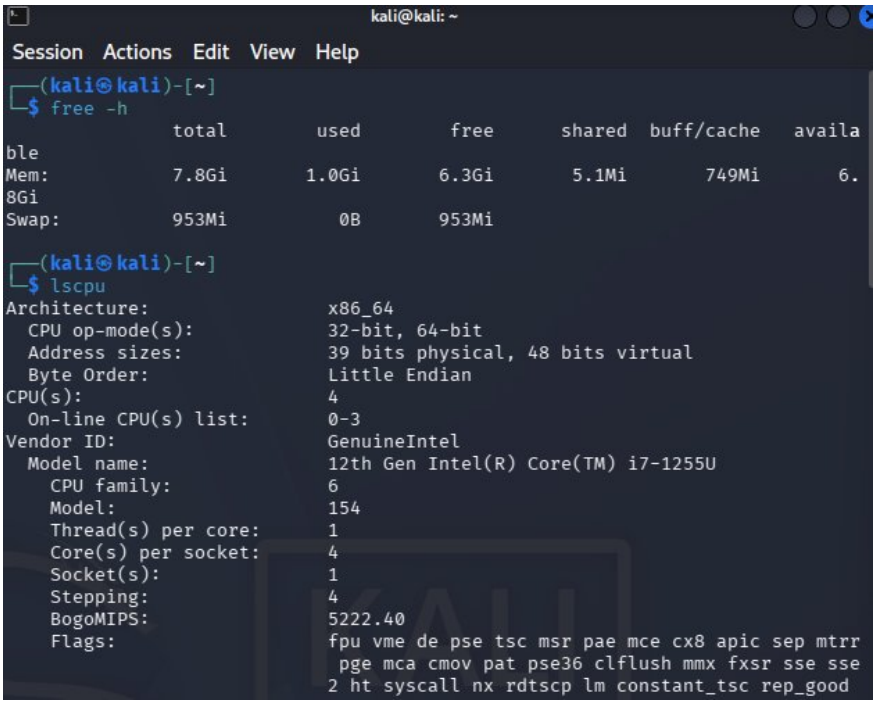


Figura 1: *Comandos `free -h` y `lscpu` en la máquina virtual Kali.*

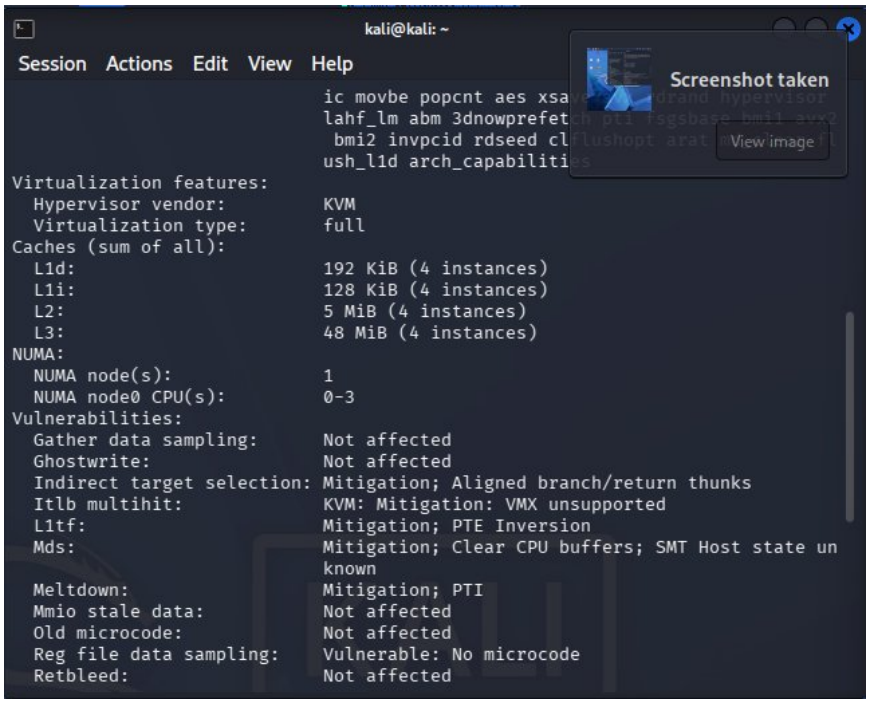


Figura 2: *Continuación de `lscpu`: virtualización KVM y memoria caché.*

Tabla 3: *Recursos del equipo físico y de la máquina virtual.*

Componente	Computador físico	Máquina virtual (kali clon)
Procesador	Intel Core i7-1255U (12.ª gen.)	4 núcleos asignados, con instrucciones AVX2
RAM	16 GB	7,8 GB en total (6,8 GB disponibles)
GPU / VRAM	Gráficos integrados Intel	VMware SVGA II (adaptador virtual, sin aceleración)
Almacenamiento	Disco del portátil	79 GB, de los cuales 54 GB libres
Virtualización	VirtualBox	KVM, virtualización completa

Conclusión del punto 3: como la máquina virtual no tiene acceso a la tarjeta gráfica, los modelos se ejecutan únicamente con el procesador, apoyándose en las instrucciones AVX2. Con 6,8 GB de RAM disponibles son viables los modelos de 1B a 4B de parámetros. Un modelo de 8B (unos 5 GB) podría cargar, pero dejaría muy poca memoria para el sistema y Wireshark y respondería muy lento. Por eso se eligieron **Llama 3.2 3B** y **Qwen 3 4B**.

4. Selección e implementación de los modelos

4.1. Modelos seleccionados

Tabla 4: *Modelos elegidos.*

Modelo	Parámetros	Tamaño	Motivo de la elección
Llama 3.2 3B (Meta)	3.000 millones	2,0 GB	El más liviano y rápido; sirve como punto de comparación.
Qwen 3 4B (Alibaba)	4.000 millones	2,5 GB	Un poco más grande y con modo de razonamiento (*thinking*), lo que debería dar mejores análisis.

4.2. Instalación de Ollama

El instalador automático (`curl -fsSL https://ollama.com/install.sh | sh`) se cortó al 94,9 % por una caída de la conexión. Por eso Ollama se descargó manualmente con `wget -c`, que permite reanudar la descarga si se interrumpe, y se descomprimió en el sistema:

```
wget -c https://ollama.com/download/ollama-linux-amd64.tar.zst
sudo tar --zstd -xf ollama-linux-amd64.tar.zst -C /usr/local
```

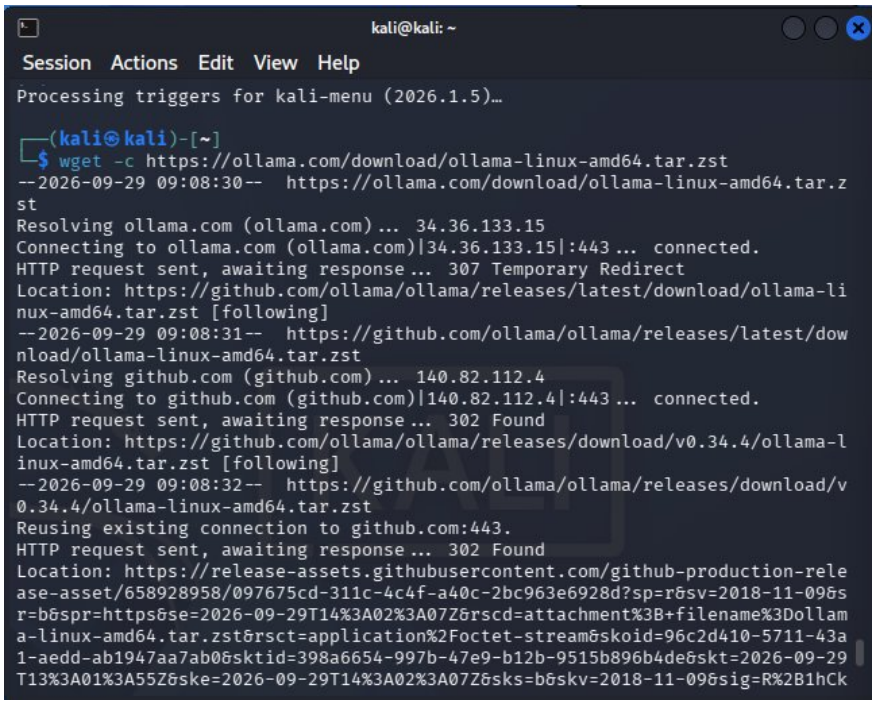


Figura 3: *Descarga manual de Ollama con* `wget -c`.

Luego se verificó la versión (0.34.4) y se descargaron los dos modelos con `ollama pull`. El comando `ollama list` confirma que quedaron instalados.

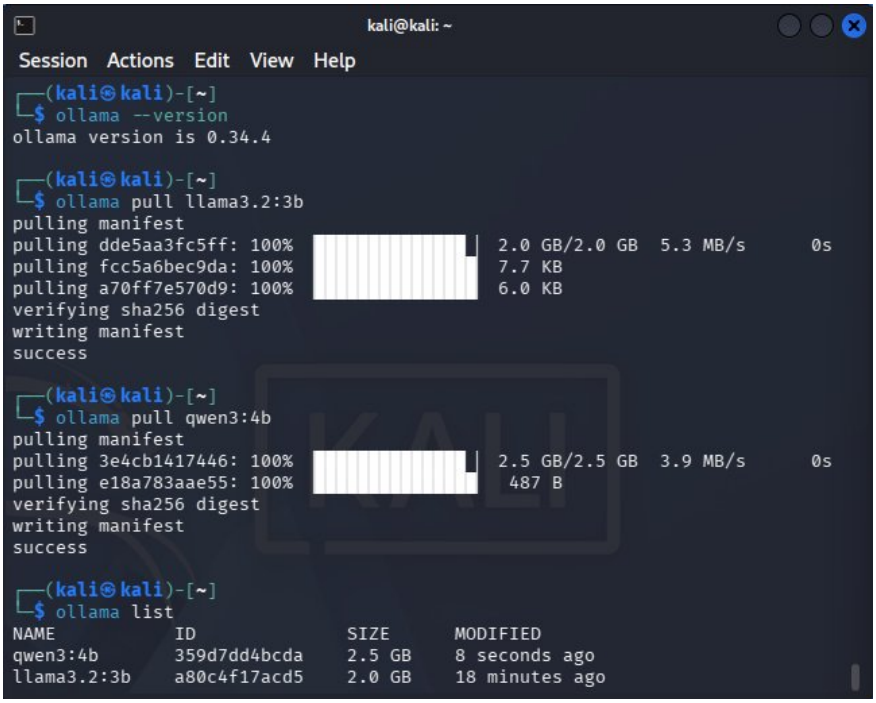


Figura 4: *Versión de Ollama, descarga de los dos modelos y* `ollama list`.

Como Ollama se instaló a mano, el servicio se debe arrancar en una segunda terminal con `ollama serve`, que debe permanecer abierta mientras se usan los modelos.

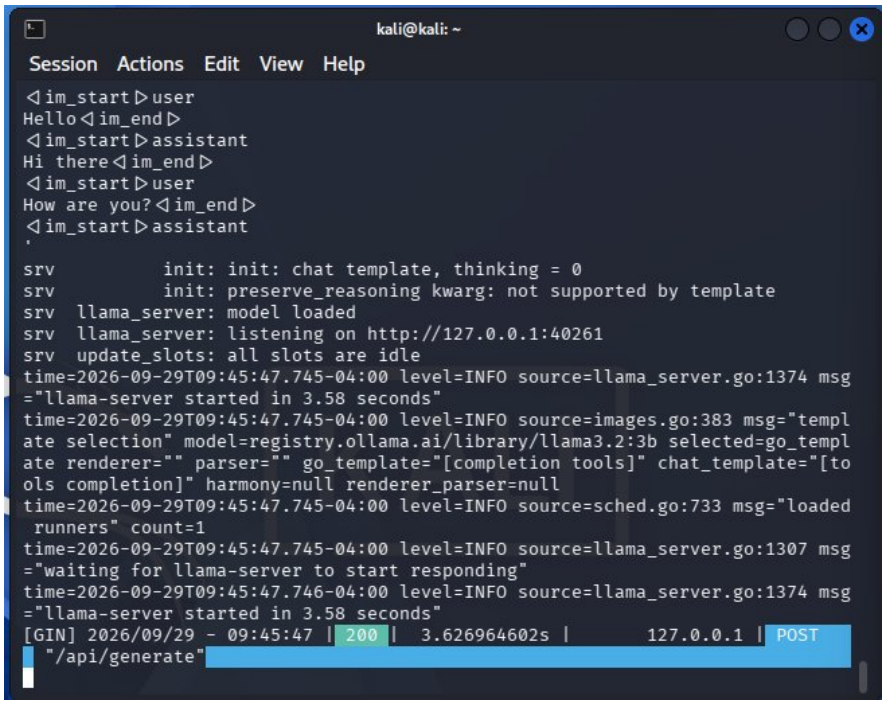


Figura 5: *Servicio `ollama serve` funcionando en la segunda terminal.*

4.3. Pruebas de funcionamiento

Cada modelo se probó con `ollama run <modelo> -verbose`, que al final de cada respuesta muestra estadísticas de tiempo y velocidad. Primero se hizo la pregunta “Explica la diferencia entre el tráfico SSH y el tráfico de una VPN” y luego una pregunta larga (“Explica detalladamente cómo funciona el protocolo SSH, paso a paso”) para medir el consumo de recursos con `ollama ps` y `htop`.

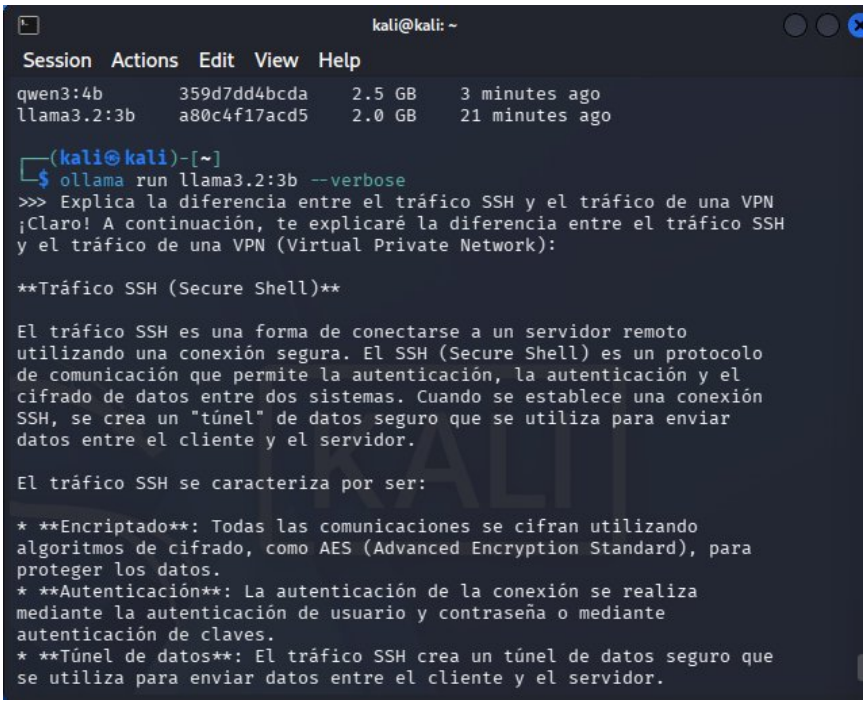


Figura 6: *Primera pregunta a Llama 3.2 3B.*

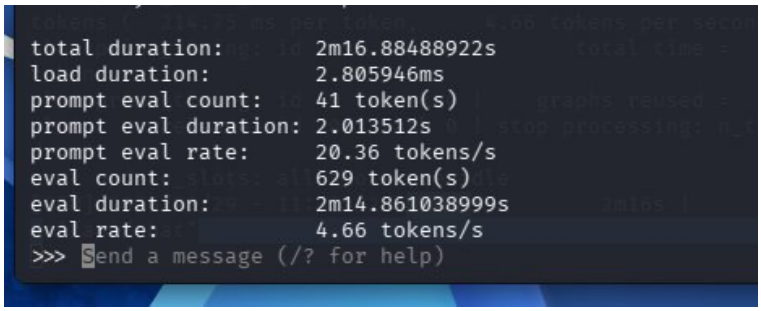


Figura 7: *Estadísticas de la primera pregunta en Llama 3.2 3B.*

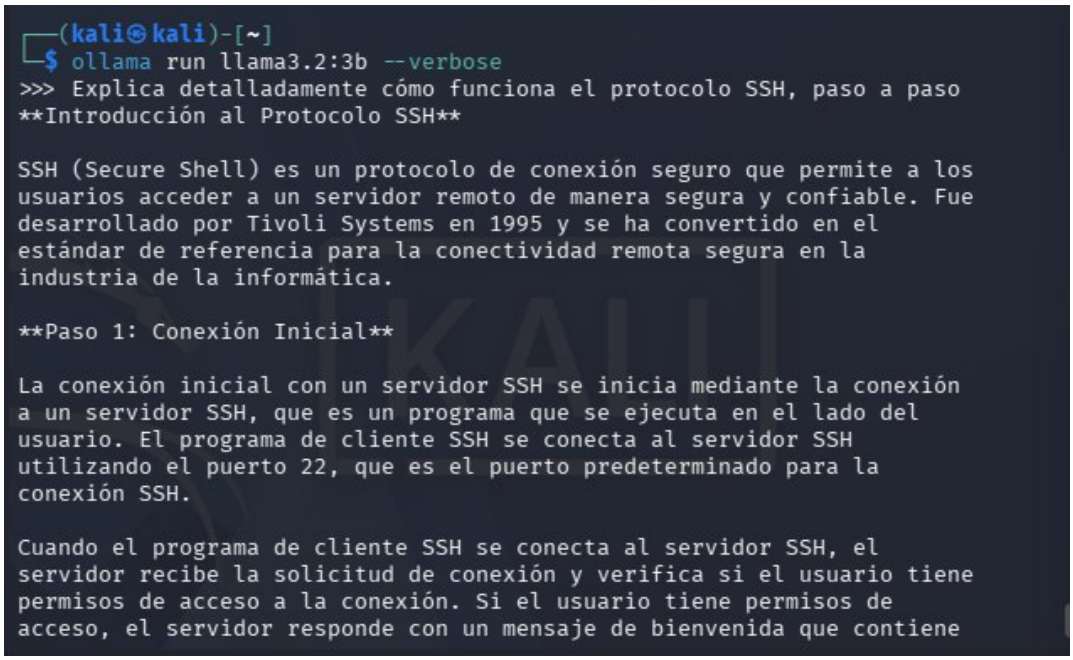


Figura 8: *Pregunta larga a Llama 3.2 3B para medir el consumo de recursos.*

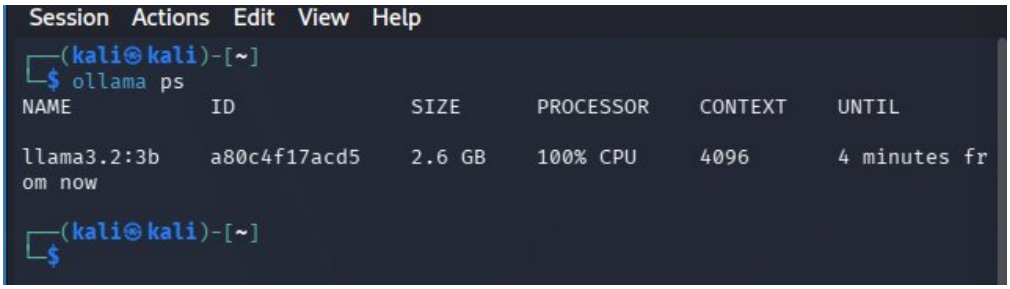


Figura 9: *`ollama ps` con Llama 3.2 3B: 2,6 GB en memoria y 100 % CPU.*

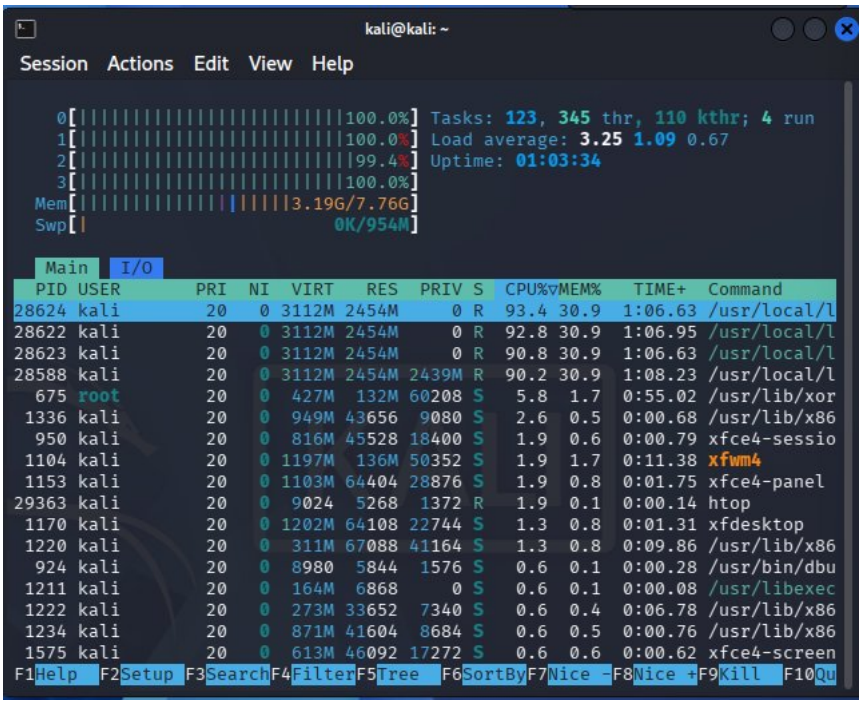


Figura 10: *`htop` mientras responde Llama 3.2 3B.*

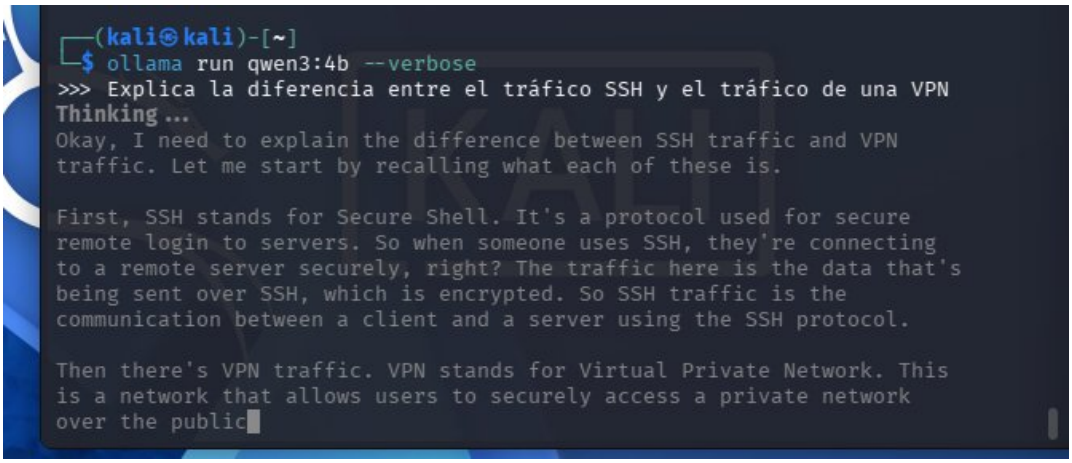


Figura 11: *Qwen 3 4B: primero muestra su bloque de razonamiento (Thinking).*

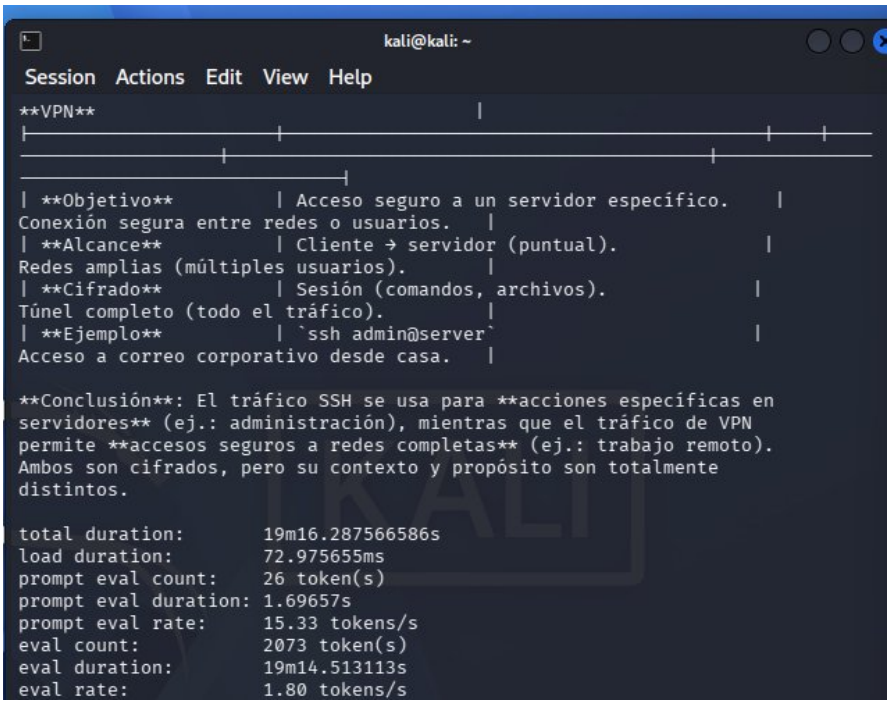


Figura 12: *Respuesta y estadísticas de la primera pregunta en Qwen 3 4B.*

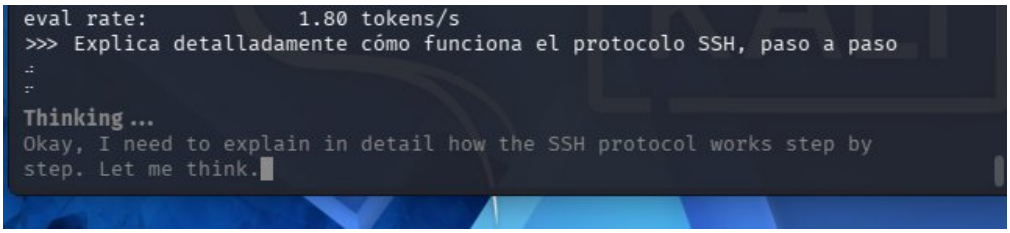


Figura 13: *Pregunta larga a Qwen 3 4B.*

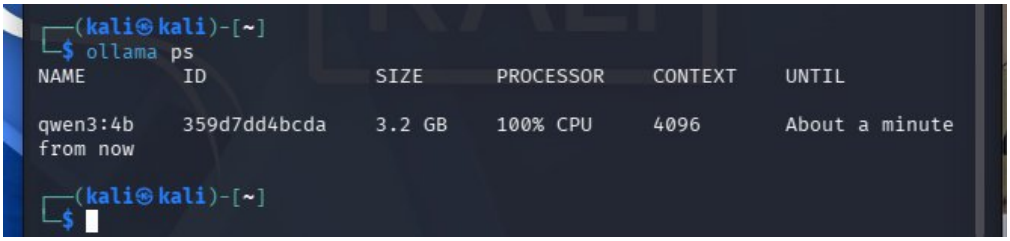


Figura 14: `ollama ps` *con Qwen 3 4B: 3,2 GB en memoria y 100 % CPU.*

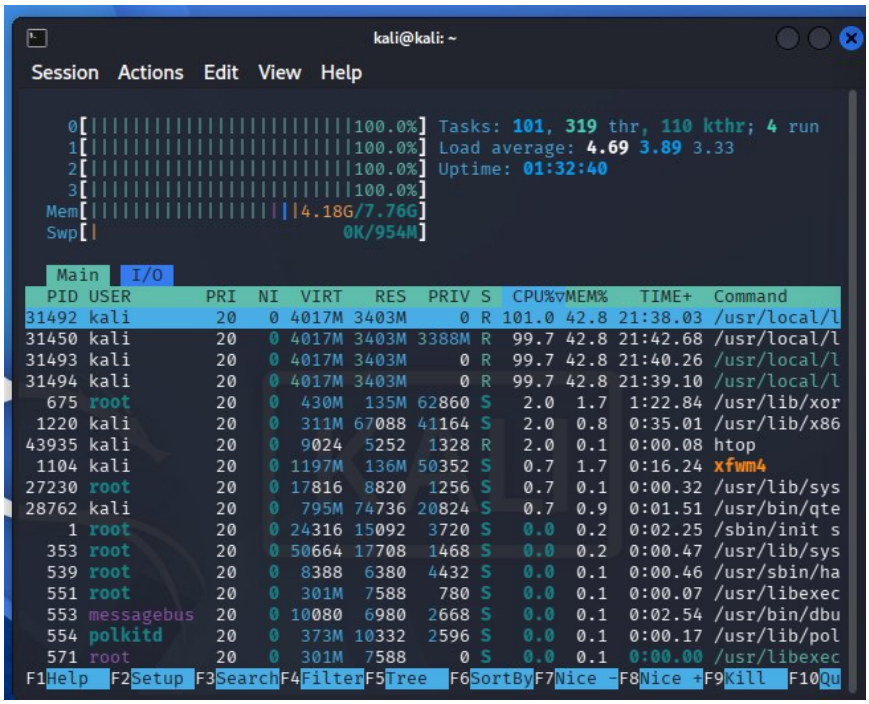


Figura 15: *`htop` mientras responde Qwen 3 4B.*

Tabla 5: *Resultados de las pruebas de funcionamiento.*

Dato	**Llama 3.2 3B**	**Qwen 3 4B**
Tiempo total (primera pregunta)	2 min 16 s	19 min 16 s
Tokens generados	629	2.073 (incluye el razonamiento)
Velocidad de generación	4,66 tokens/s	1,80 tokens/s
Memoria según `ollama ps`	2,6 GB	3,2 GB
RAM del proceso en `htop` (RES)	≈ 2,4 GB (30,9 %)	≈ 3,3 GB (42,8 %)
Uso de CPU	100 % en los 4 núcleos	100 % en los 4 núcleos
Carga promedio (*load average*)	3,25	4,69
Memoria swap usada	0	0

Los dos modelos funcionan, pero Qwen tarda unas 8 veces más en contestar la misma pregunta: tiene un 33 % más de parámetros y además “piensa” antes de responder, lo que genera muchos más tokens. Ninguno usó swap, así que los 8 GB asignados alcanzan para ambos.

4.4. Problemas encontrados y solución

Tabla 6: *Problemas durante la instalación.*

Problema	Causa	Solución
El instalador de Ollama falló al 94,9 %	Corte de la conexión a Internet	Descarga manual con `wget -c`, que reanuda donde quedó.
Error "llama-server binary not found"	El primer intento dejó una instalación incompleta en /usr/local	Descomprimir de nuevo el paquete en /usr/local.
No funcionaba copiar y pegar en Ubuntu	Ubuntu usaba Wayland, incompatible con el portapapeles de VirtualBox	Se cambió a Kali Linux, que además ya trae Wireshark.

5. Aplicación de la IA al análisis de tráfico SSH y VPN

5.1. Preparación del escenario

Como no se tenían las capturas del laboratorio, se volvió a montar el escenario con dos máquinas Kali conectadas por una red de solo anfitrión de VirtualBox (cada una con un segundo adaptador de red):

Tabla 7: *Roles de las máquinas virtuales.*

Máquina	IP (solo anfitrión)	IP del túnel	Rol
kali clon	192.168.56.106	10.10.10.1	Servidor SSH y WireGuard; aquí se capturó el tráfico y se ejecutó la IA
kali	192.168.56.104	10.10.10.2	Cliente

Captura SSH. En el servidor se encendió SSH y se capturó solo el puerto 22 con `sudo tshark -i eth1 -f "tcp port 22" -w /tmp/ssh.pcapng` (se guardó en /tmp porque tshark con sudo no podía escribir en la carpeta personal). Desde el cliente se abrió la sesión `ssh kali@192.168.56.106` y se ejecutaron `ls -la`, `whoami` y `cat /etc/os-release`. Se capturaron 3.248 paquetes.

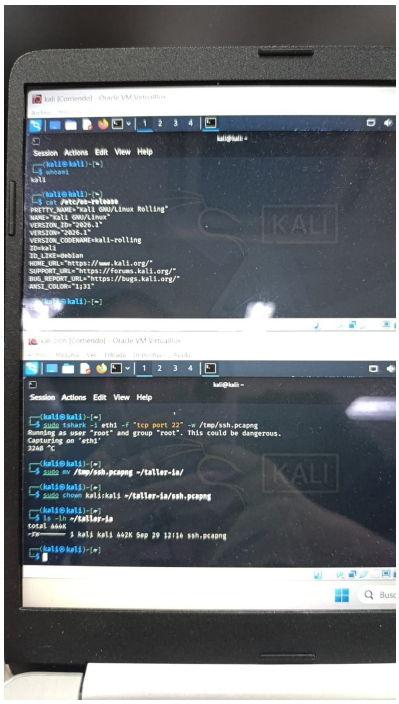


Figura 16: *Captura SSH: comandos del cliente (arriba) y* `tshark` *en el servidor (abajo).*

Montaje de la VPN. En las dos máquinas se instaló `wireguard-tools`, se generó un par de claves con `wg genkey | tee privada.key | wg pubkey > publica.key` y se intercambiaron las claves públicas con `scp`. Luego se creó el archivo `/etc/wireguard/wg0.conf` de cada máquina (el servidor escucha en el puerto UDP 51820).

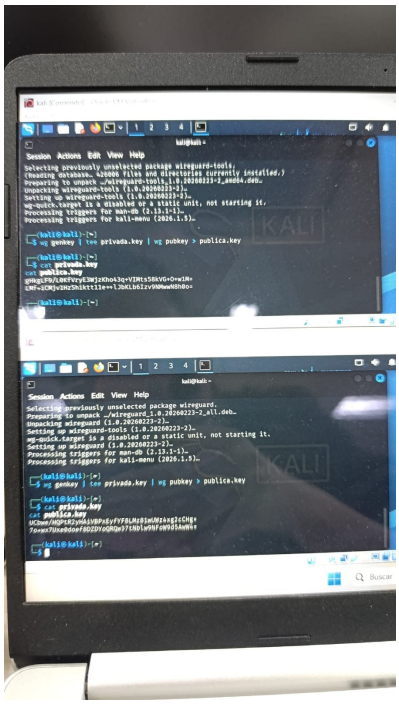


Figura 17: *Instalación de WireGuard y generación de claves en ambas máquinas.*

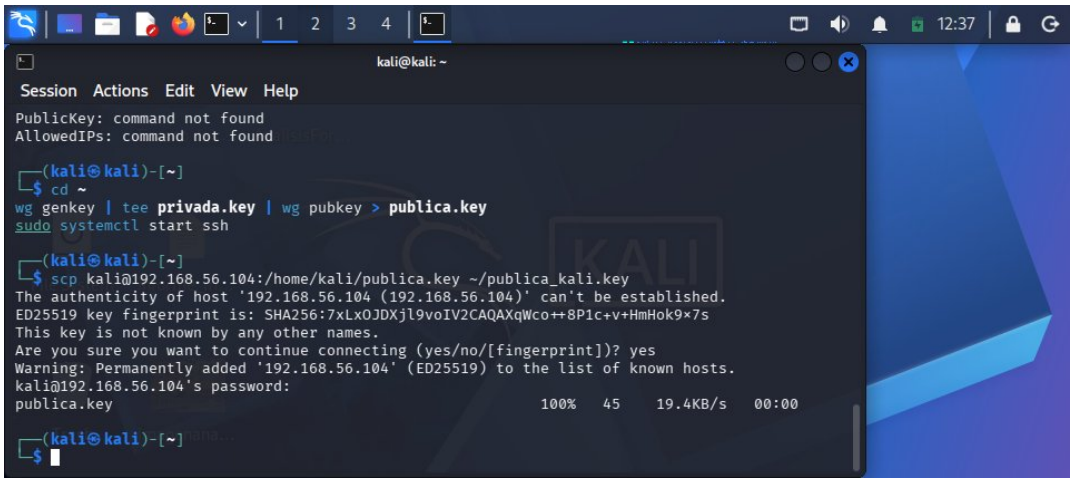


Figura 18: *kali clon obtiene la clave pública del cliente con* `scp`.

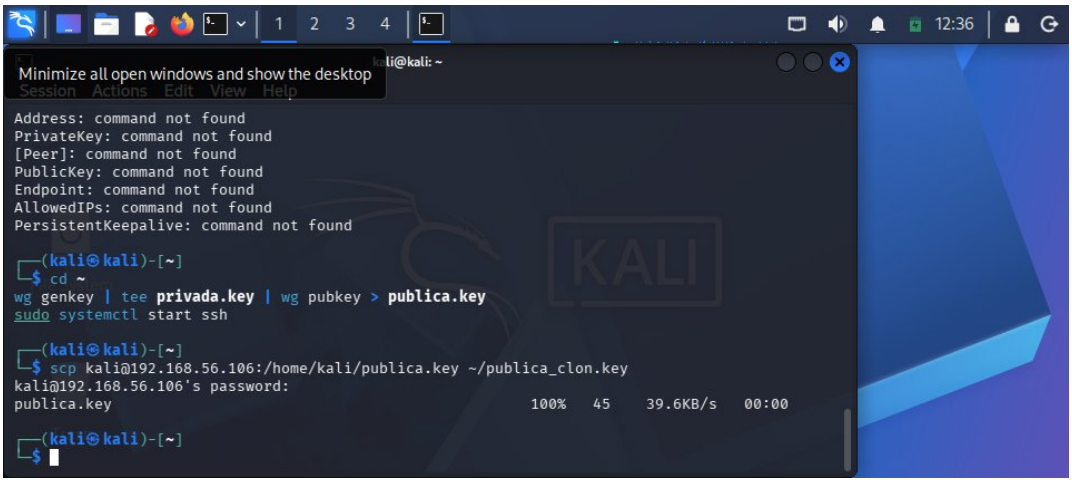


Figura 19: *kali obtiene la clave pública del servidor con* `scp`.

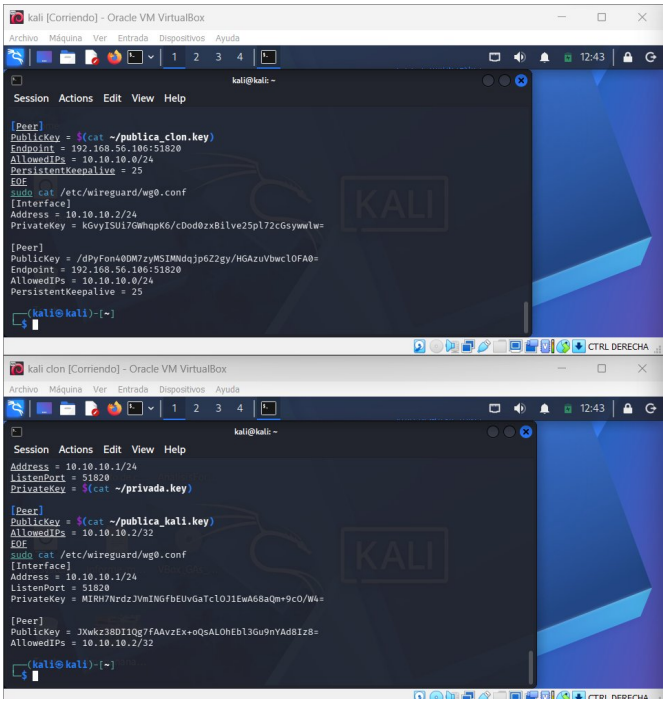


Figura 20: *Archivos* `wg0.conf` *del cliente (arriba) y del servidor (abajo).*

Captura VPN. En el servidor se capturó todo el tráfico de la interfaz con `sudo tshark -i eth1 -w /tmp/vpn.pcapng` y se levantó el túnel con `sudo wg-quick up wg0` en ambas máquinas. Desde el cliente se hizo `ping -c 5 10.10.10.1` y una sesión **SSH por dentro del túnel** (`ssh kali@10.10.10.1`). El comando `sudo wg` muestra "latest handshake: Now", lo que confirma que la VPN quedó establecida.

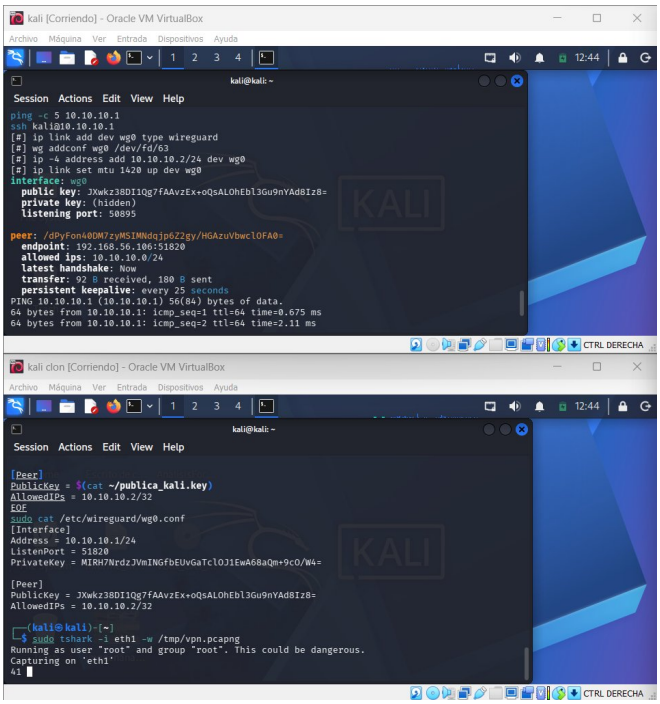


Figura 21: *Túnel establecido (latest handshake), ping por la VPN y captura en curso.*

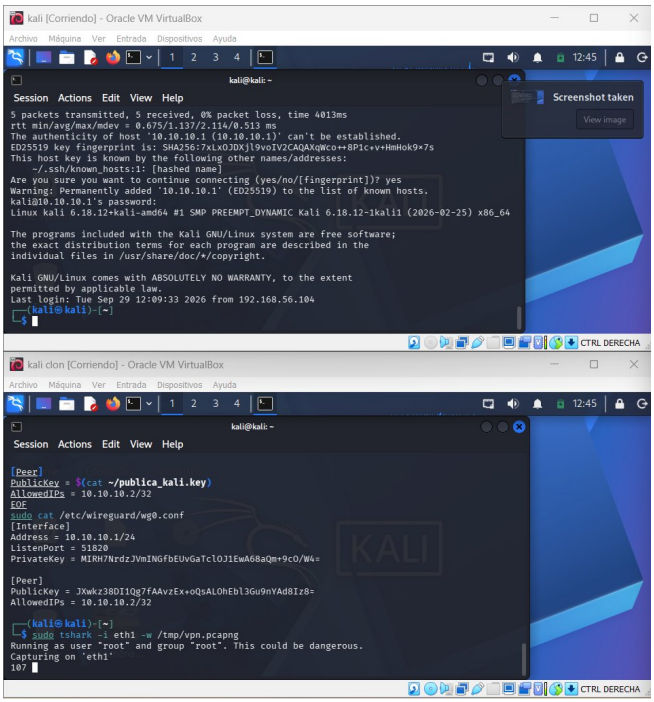


Figura 22: *Sesión SSH dentro del túnel mientras se captura el tráfico.*

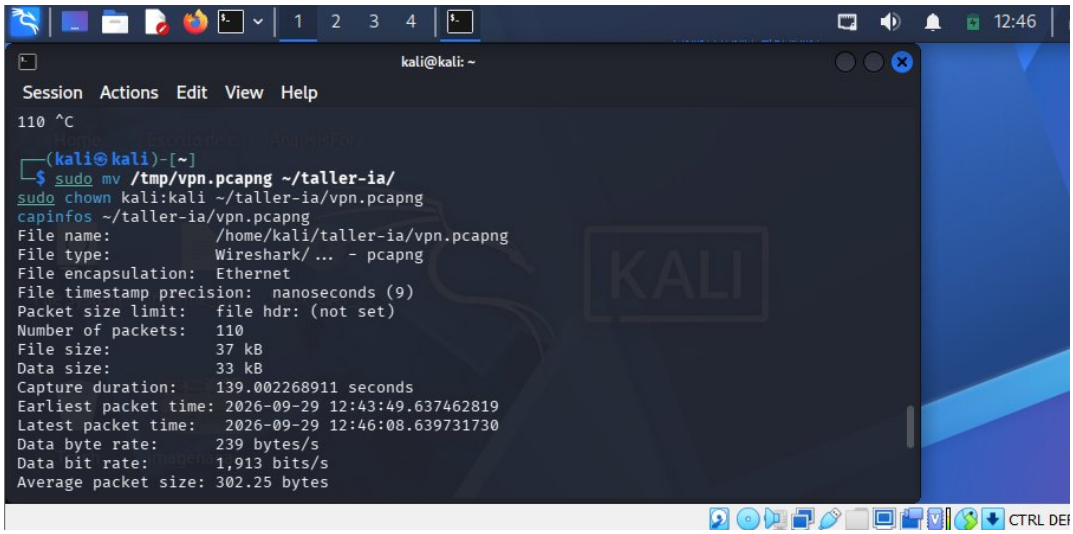


Figura 23: *capinfos de vpn.pcapng: 110 paquetes en 139 segundos.*



Figura 24: *Cierre del túnel con wg-quick down.*

5.2. Preparación de la información para la IA

Los modelos de lenguaje no pueden abrir un archivo `.pcapng`; solo entienden texto. Por eso, con `tshark`, cada captura se convirtió en un **resumen de texto** con la jerarquía de protocolos, las conversaciones IP y los primeros 40 paquetes (654 y 667 palabras). Además, Ollama se arrancó con `OLLAMA_CONTEXT_LENGTH=8192` para que los modelos pudieran leer textos más largos.

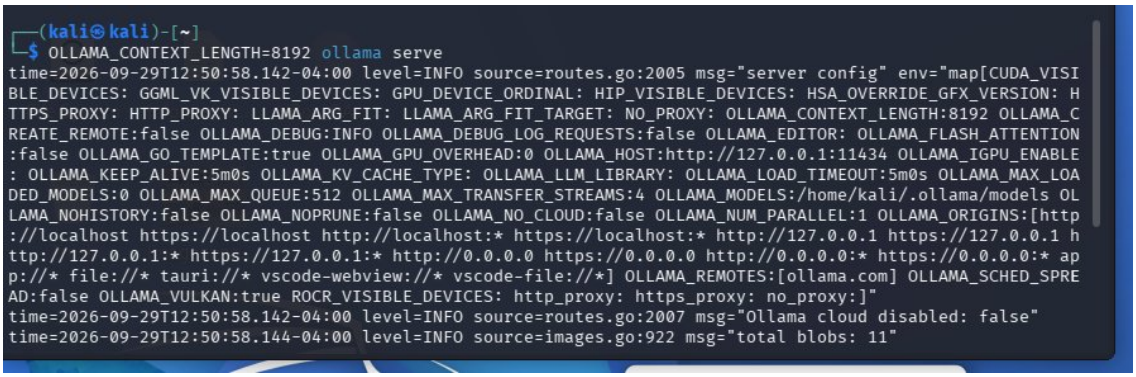


Figura 25: *Ollama iniciado con un contexto de 8.192 tokens.*

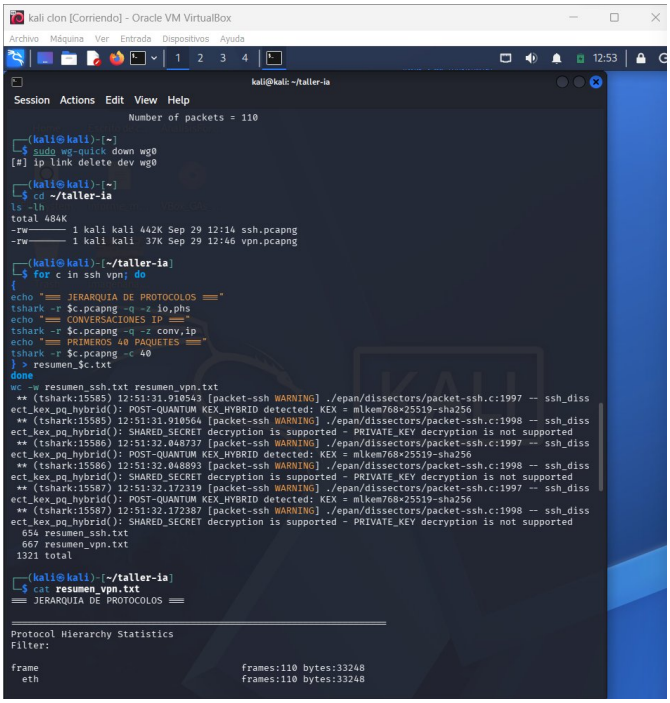


Figura 26: *Generación de los resúmenes de texto con* `tshark`.



Figura 27: *Contenido de uno de los resúmenes.*

Con cada resumen se armó un archivo de instrucciones (*prompt*) con las mismas cinco preguntas para los dos modelos: protocolos, direcciones IP y roles, patrones de comunicación, anomalías, e información visible y cifrada. También se creó un tercer prompt con los dos resúmenes para que compararan SSH contra VPN. Usar exactamente el mismo texto en ambos modelos es lo que hace justa la comparación.

```
┌──(kali㉿kali)-[~/taller-ia]
└─$ P="Eres un analista de ciberseguridad. A continuación tienes un resumen de una captura de tráfico d
e red obtenida con tshark. Analízala y responde en español:
1. ¿Qué protocolos aparecen y cuál es el principal?
2. ¿Qué direcciones IP participan y cuál parece ser el cliente y cuál el servidor?
3. ¿Qué patrones de comunicación observas (puertos, dirección del tráfico, intercambio inicial)?
4. ¿Hay posibles anomalías o comportamientos sospechosos?
5. ¿Qué información se puede ver y qué información está cifrada?
"
{ echo "$P"; cat resumen_ssh.txt; } > prompt_ssh.txt
{ echo "$P"; cat resumen_vpn.txt; } > prompt_vpn.txt
{
echo "Eres un analista de ciberseguridad. Tienes dos resúmenes de capturas de tráfico: una de una conex
ión SSH y otra de una conexión VPN. Compáralas y explica en español las diferencias en protocolos, puer
tos, patrones de comunicación, cifrado y visibilidad de la información."
echo "##### CAPTURA SSH #####"; cat resumen_ssh.txt
echo "##### CAPTURA VPN #####"; cat resumen_vpn.txt
} > prompt_comparacion.txt
ls
prompt_comparacion.txt  prompt_vpn.txt   resumen_vpn.txt  vpn.pcapng
prompt_ssh.txt          resumen_ssh.txt  ssh.pcapng

┌──(kali㉿kali)-[~/taller-ia]
└─$
```

Figura 28: *Creación de los archivos de prompt.*

Cada análisis se ejecutó así, guardando la respuesta y las estadísticas en un archivo:

```
ollama run llama3.2:3b --verbose < prompt_ssh.txt 2>&1 | tee llama_ssh.txt
```

5.3. Resultados de Llama 3.2 3B

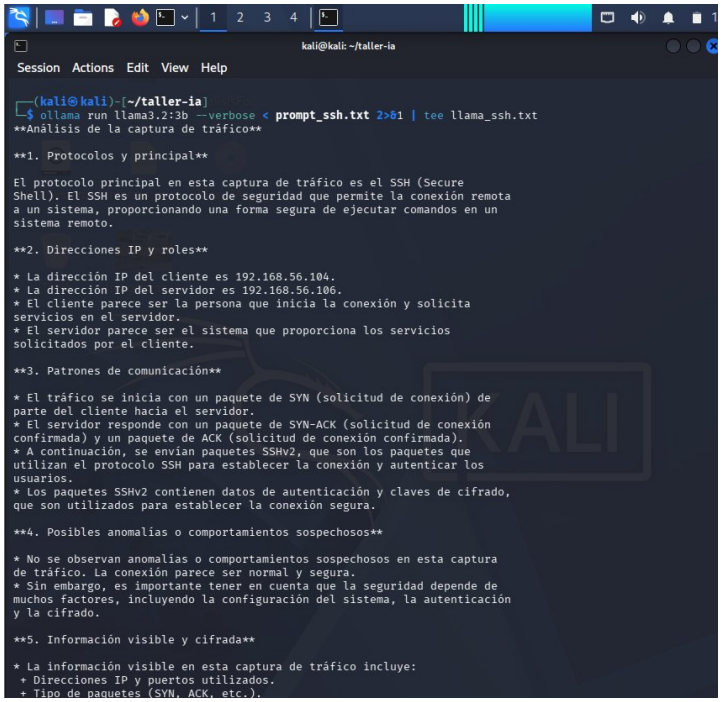


Figura 29: *Análisis 1: Llama 3.2 3B sobre la captura SSH.*

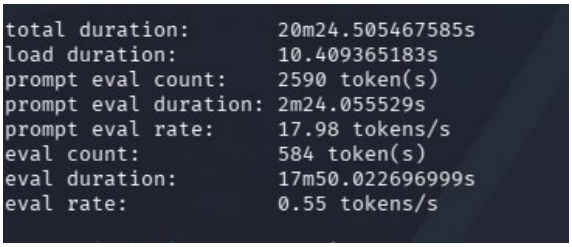


Figura 30: *Estadísticas del análisis 1.*

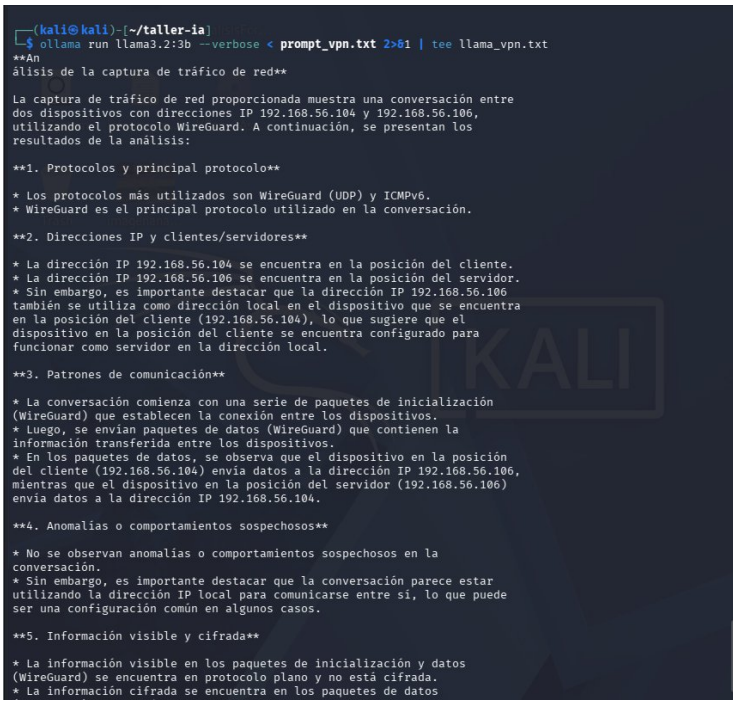


Figura 31: *Análisis 2: Llama 3.2 3B sobre la captura VPN.*

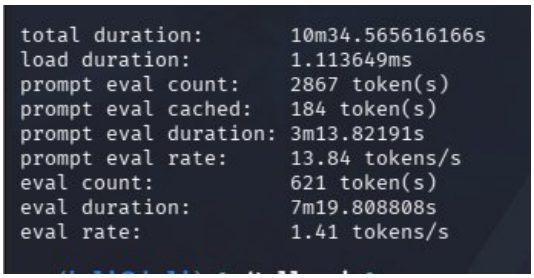


Figura 32: *Estadísticas del análisis 2.*



Figura 33: *Análisis 3: Llama 3.2 3B comparando SSH y VPN, con estadísticas.*

5.4. Resultados de Qwen 3 4B

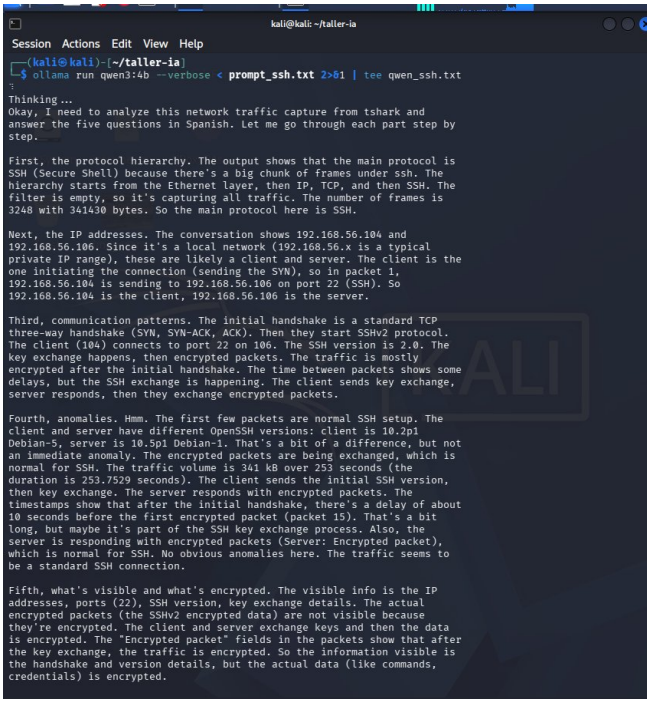


Figura 34: *Análisis 4: Qwen 3 4B sobre la captura SSH.*



Figura 35: *Respuesta final de Qwen 3 4B sobre SSH.*

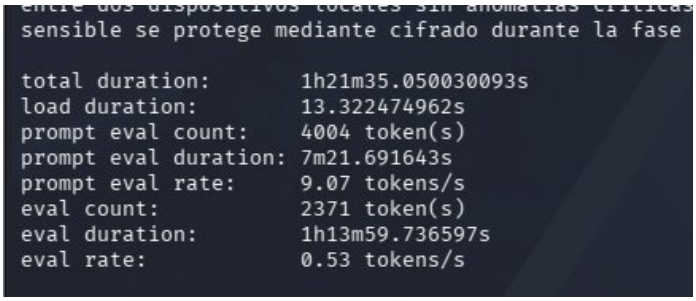


Figura 36: *Estadísticas del análisis 4.*

```
┌──(kali㉿kali)-[~/taller-ia]
└─$ ollama run qwen3:4b --verbose --think=false< prompt_vpn.txt 2>&1 | tee qwen_vpn.txt
Okay, I need to analyze this network traffic capture from tshark and
answer the five questions in Spanish. Let me go through each part step by
step.

First, the user wants to know the protocols that appear and the main one.
Looking at the hierarchy statistics, the top-level protocols are Ethernet,
IP (both IPv4 and IPv6), UDP, and ARP. The main protocol here seems to be
UDP because there are 80 frames (out of 110 total) using UDP. The data
shows a lot of UDP traffic for MDNS, WireGuard, XML, DHCP, etc.

Next, the IP addresses involved. From the IPv4 conversations section, we
have 192.168.56.104 <-> 192.168.56.106, 192.168.56.1 with multicast
addresses (224.0.0.251, 239.255.255.250, 255.255.255.255). The client and
server: in the WireGuard handshake (packets 9-10), 192.168.56.104 is
initiating to 192.168.56.106. So the client is probably 192.168.56.104 and
server 192.168.56.106. The 192.168.56.1 might be a local machine (like a
router or host) doing MDNS queries.

Third, communication patterns: the MDNS queries for _dosvc._tcp.local,
which is typical for service discovery. Then WireGuard handshake between
104 and 106, which is a secure tunnel. Also, ARP exchanges in the last few
packets (38-40) between two MAC addresses. The initial traffic shows MDNS
queries from 192.168.56.1 to the multicast groups.

Fourth, anomalies: The WireGuard traffic has a lot of data packets
(counter up to 14) with varying data lengths. The initial handshake is
normal, but the high volume of data packets might be normal for WireGuard.
However, the presence of MDNS queries for a specific service (dosvc) could
be normal if it's a service discovery. The ARP exchanges seem to be part
of the network configuration. But the main anomaly might be the large
number of WireGuard packets (packets 9-39) which is a normal handshake but
maybe a bit high? Wait,
the                                                    WireGuard handshake is quick (packets
9-10)
and then keepalives. The data packets after that are for data transfer.
Maybe the 192.168.56.1 is the client (local machine) and 192.168.56.106 is
the server (WireGuard node). The MDNS queries are for the local network
service discovery, so that's normal.

Fifth, what's visible and encrypted: The MDNS queries have plaintext
(they're DNS queries), the WireGuard traffic is encrypted (WireGuard uses
encryption), but the initial handshake packets (like the 190 and 134) have
some data, but the actual data is encrypted. The XML and DHCP packets
might have plaintext info (like DHCP options), but the WireGuard traffic
is encrypted.
```

Figura 37: *Análisis 5: Qwen 3 4B sobre la captura VPN.*

```
### Análisis de la captura de tráfico de red con tshark

**1. ¿Qué protocolos aparecen y cuál es el principal?**
Los protocolos detectados son: **Ethernet**, **IP (IPv4 y IPv6)**,
**UDP**, **MDNS**, **WireGuard**, **DHCP**, **XML**, **ARP** y **ICMPv6**.

El **principal protocolo** es **UDP** (80 de 110 frames, 24.854 bytes), ya
que representa el 73% del tráfico y se usa para servicios como MDNS,
WireGuard y DHCP.

---

**2. ¿Qué direcciones IP participan y cuál parece ser el cliente y cuál el
servidor?**
- **Direcciones IP clave**:
  - `192.168.56.104` (cliente, inicia la conexión WireGuard)
  - `192.168.56.106` (servidor, responde al cliente en WireGuard)
  - `192.168.56.1` (cliente, realiza consultas MDNS)
  - Multicast (ej: `224.0.0.251`, `239.255.255.250`, `255.255.255.255`)
  - IPv6 (ej: `fe80::12eb:b6a4:732:2cc4`, `ff02::fb`)

- **Cliente y servidor**:
  - **Cliente**: `192.168.56.104` (inicia el handshake de WireGuard con
`192.168.56.106`).
  - **Servidor**: `192.168.56.106` (responde al cliente en WireGuard).
  - *Nota*: `192.168.56.1` también actúa como cliente para descubrir
servicios (MDNS).

---

**3. ¿Qué patrones de comunicación observas (puertos, dirección del
tráfico, intercambio inicial)?**
- **Patrones clave**:
  1. **Descubrimiento de servicios MDNS**:
     - Consultas para `_dosvc._tcp.local` desde `192.168.56.1` a multicast
(`224.0.0.251`).
     - Respuestas con direcciones IP y MAC (ej:
`DESKTOP-5LMODDP._dosvc._tcp.local` en `192.168.56.1`).
  2. **WireGuard**:
     - Handshake inicial (paquetes 9-10): `192.168.56.104` →
`192.168.56.106` (puerto 51820 por defecto).
     - Intercambio de datos encriptados (paquetes 12-39): Transferencia de
datos con *counter* y *datalen* (ej: 96 bytes por paquete).
  3. **ARP**:
     - Resolución de direcciones MAC en la red local (paquetes 38-40:
`PCSSystemtec_37:d3:a8` ↔ `PCSSystemtec_c1:b1:14`).

---

**4. ¿Hay posibles anomalías o comportamientos sospechosos?**
- **No se observan anomalías evidentes**. El tráfico es normal para una
```

Figura 38: *Respuesta de Qwen 3 4B sobre VPN (parte 1).*



Figura 39: *Respuesta de Qwen 3 4B sobre VPN (parte 2) y estadísticas.*



Figura 40: *Análisis 6: Qwen 3 4B comparando SSH y VPN, con estadísticas.*

5.5. Resumen de lo que identificó cada modelo

Las respuestas completas están en el Anexo A. En resumen:

Tabla 8: *Lo que respondió cada modelo.*

Aspecto	Llama 3.2 3B	Qwen 3 4B
Protocolos SSH	SSH como principal; menciona SYN/ACK de TCP.	TCP y SSH; SSH es el 98 % de los paquetes (3.176).
IP y roles (SSH)	Cliente .104 y servidor .106.	Cliente .104 y servidor .106, porque .104 inicia hacia el puerto 22.
Patrones SSH	Saludo TCP y luego paquetes SSHv2.	Saludo TCP, versiones OpenSSH 10.2p1 y 10.5p1, intercambio de claves híbrido post-cuántico y datos cifrados desde el paquete 15.
Protocolos VPN	WireGuard como principal; dice que ICMPv6 es de los más usados.	Lista todos (WireGuard, MDNS, ARP, DHCP, XML, ICMPv6), pero da UDP como principal.
IP y roles (VPN)	Cliente .104 y servidor .106, pero inventa que .106 también es dirección del cliente.	Cliente .104, servidor .106 y reconoce a 192.168.56.1 (el anfitrión) haciendo MDNS.
Anomalías	Ninguna en las dos capturas.	Ninguna; explica que MDNS y ARP son tráfico normal de red local.
Visible / cifrado	Dice que las claves de cifrado son visibles y, en la VPN, que WireGuard va en texto plano y cifrado a la vez.	Versiones y pasos del intercambio visibles; sesión SSH y datos de WireGuard cifrados; MDNS, ARP y DHCP en texto plano.
Comparación SSH vs. VPN	Inventa los puertos 1194 y 10000 para WireGuard y dice que la VPN usa más paquetes.	TCP/22 frente a UDP/51820, handshake de WireGuard, X25519 y ChaCha20; mezcla el tráfico de fondo con la VPN.

6. Comparación de los modelos y contraste con Wireshark

6.1. Análisis de referencia en Wireshark

Para saber si la IA acertó, primero se analizaron las dos capturas en Wireshark. Este análisis es la "respuesta correcta" contra la que se califican los modelos.

Captura SSH.

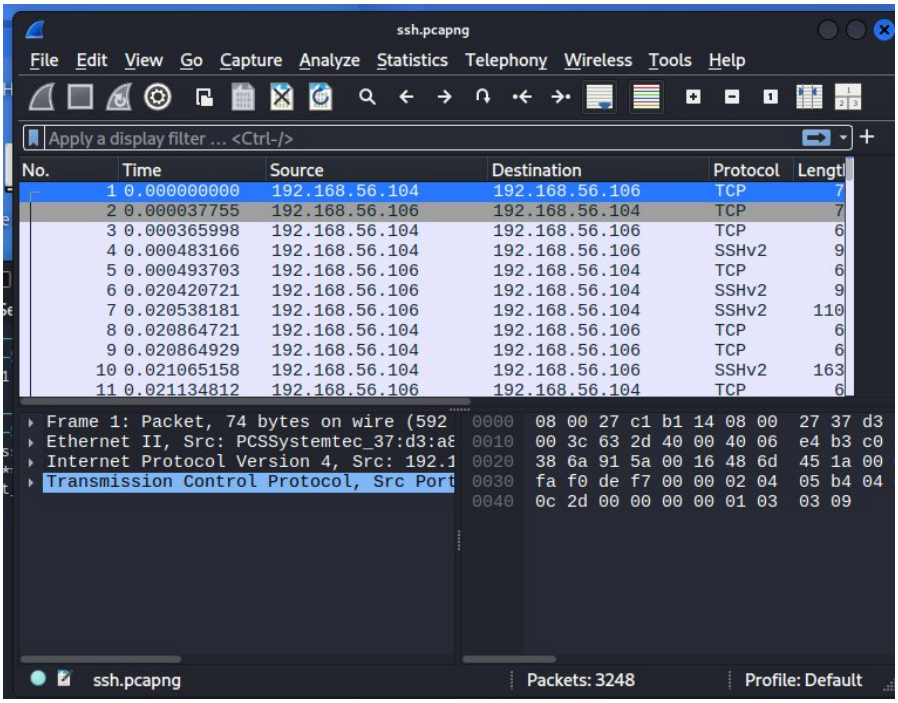


Figura 41: *ssh.pcapng abierta en Wireshark (3.248 paquetes).*

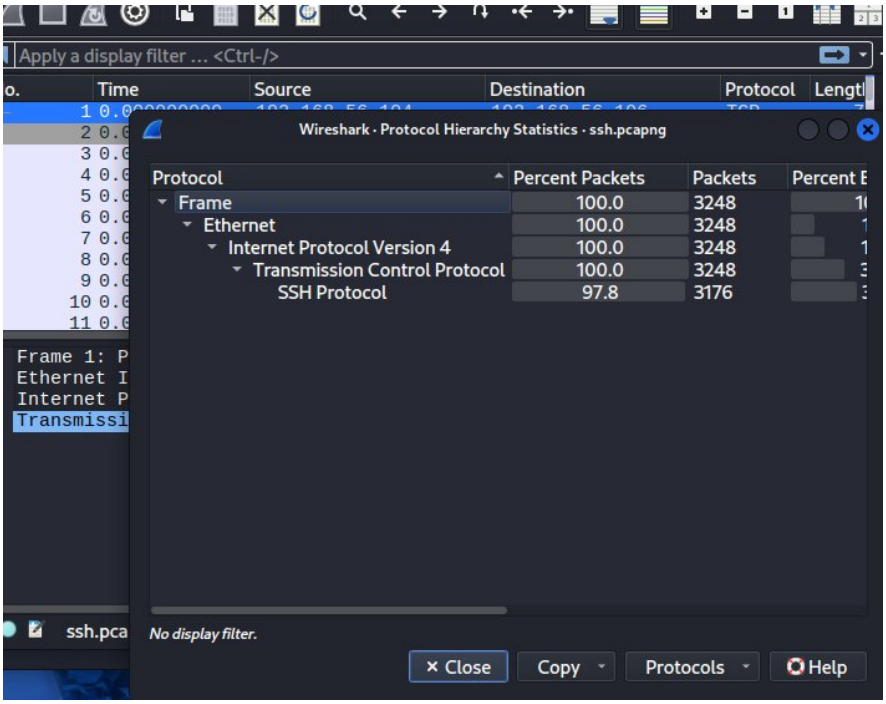


Figura 42: *Jerarquía de protocolos de la captura SSH.*

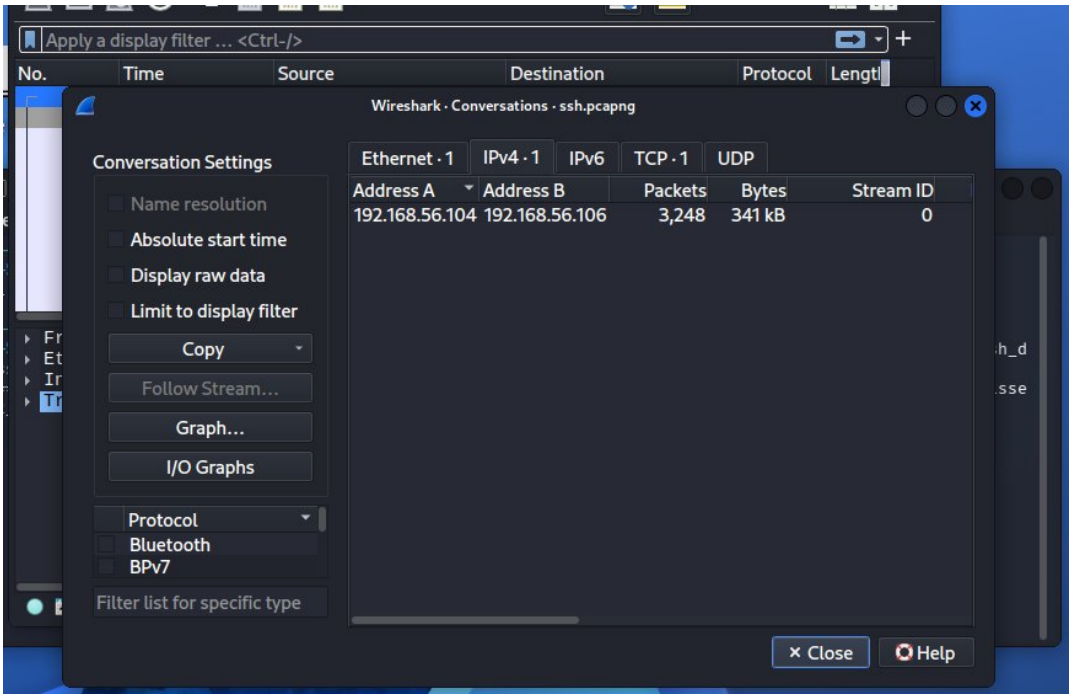


Figura 43: *Conversaciones IPv4 de la captura SSH.*

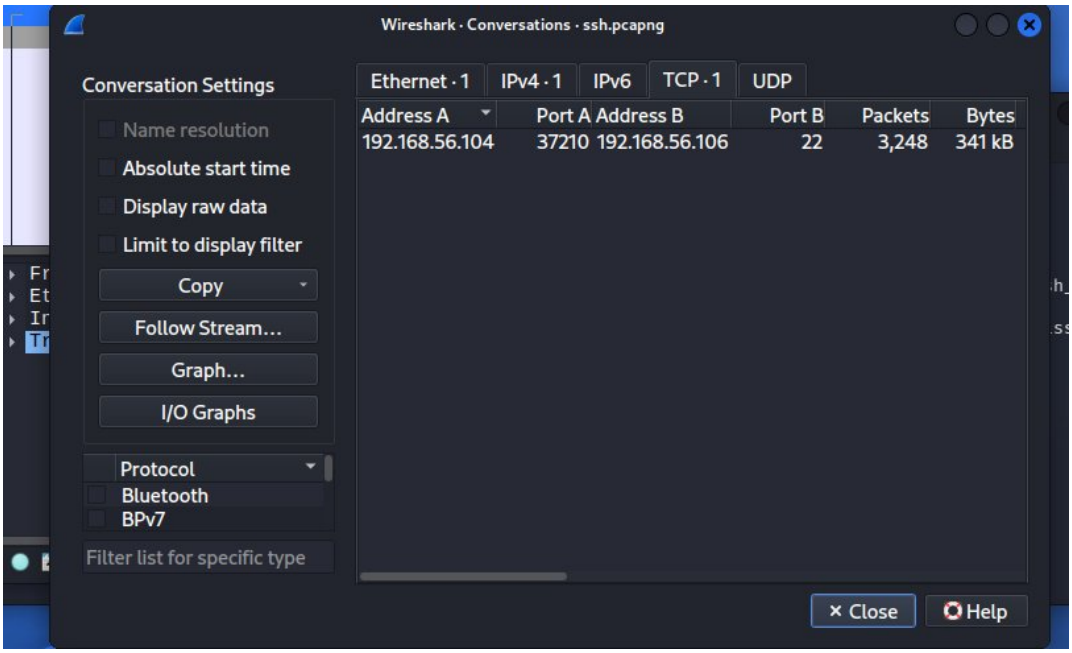


Figura 44: *Conversaciones TCP: cliente en el puerto 37210 y servidor en el 22.*

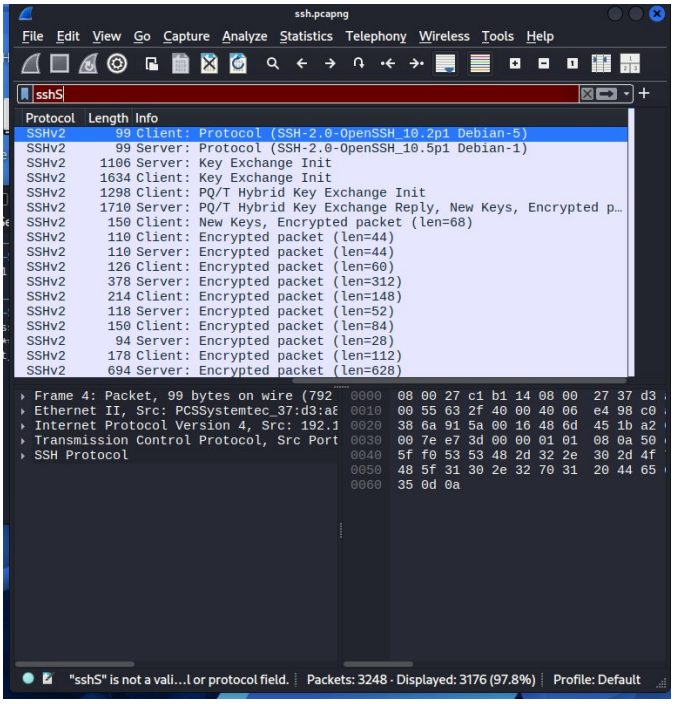


Figura 45: *Filtro `ssh`: versiones, intercambio de claves y paquetes cifrados.*

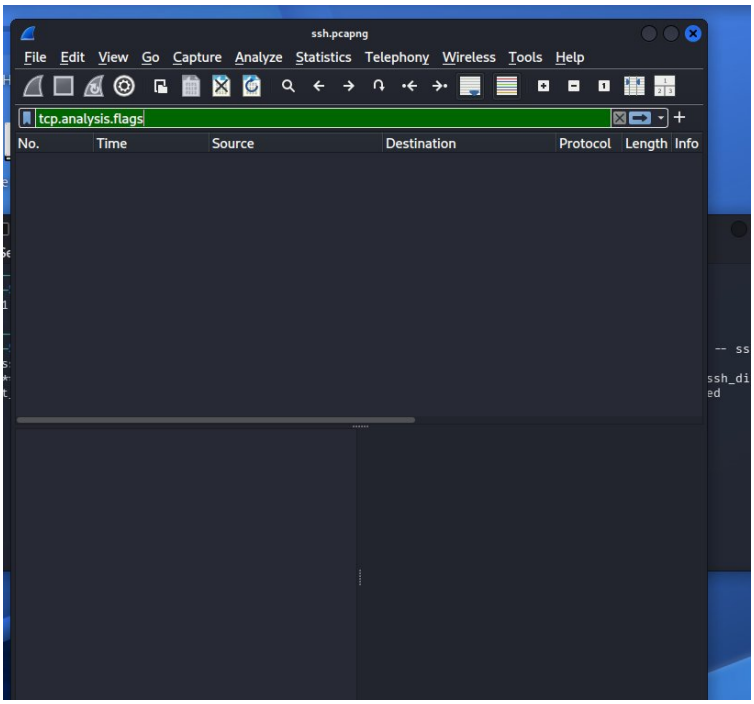


Figura 46: *Filtro `tcp.analysis.flags` vacío: no hubo retransmisiones ni errores.*

Captura VPN.

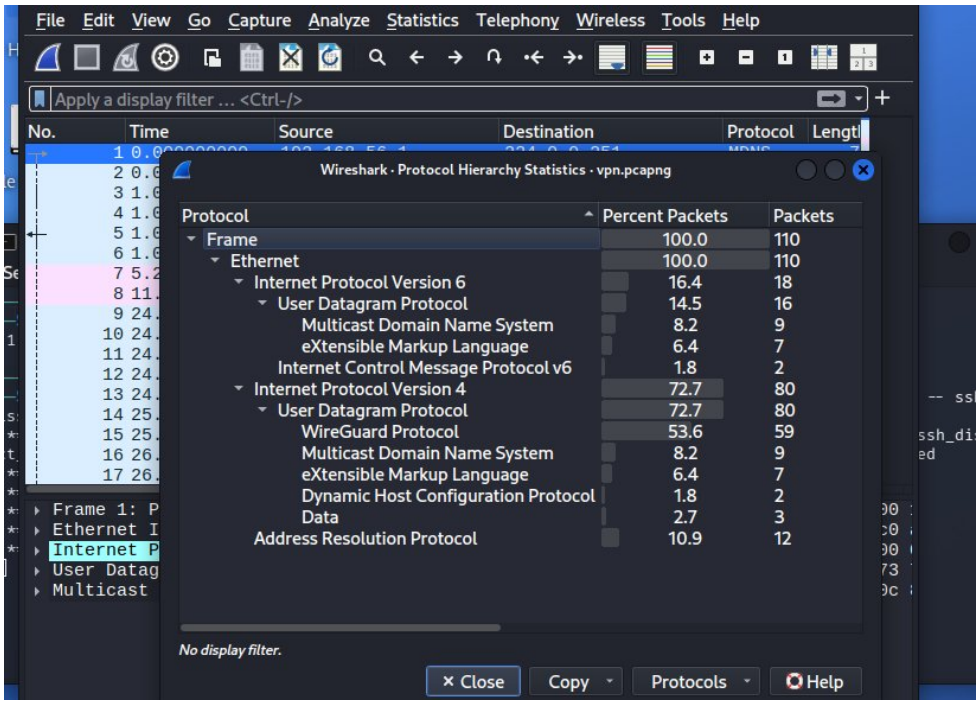


Figura 47: *Jerarquía de protocolos de la captura VPN.*

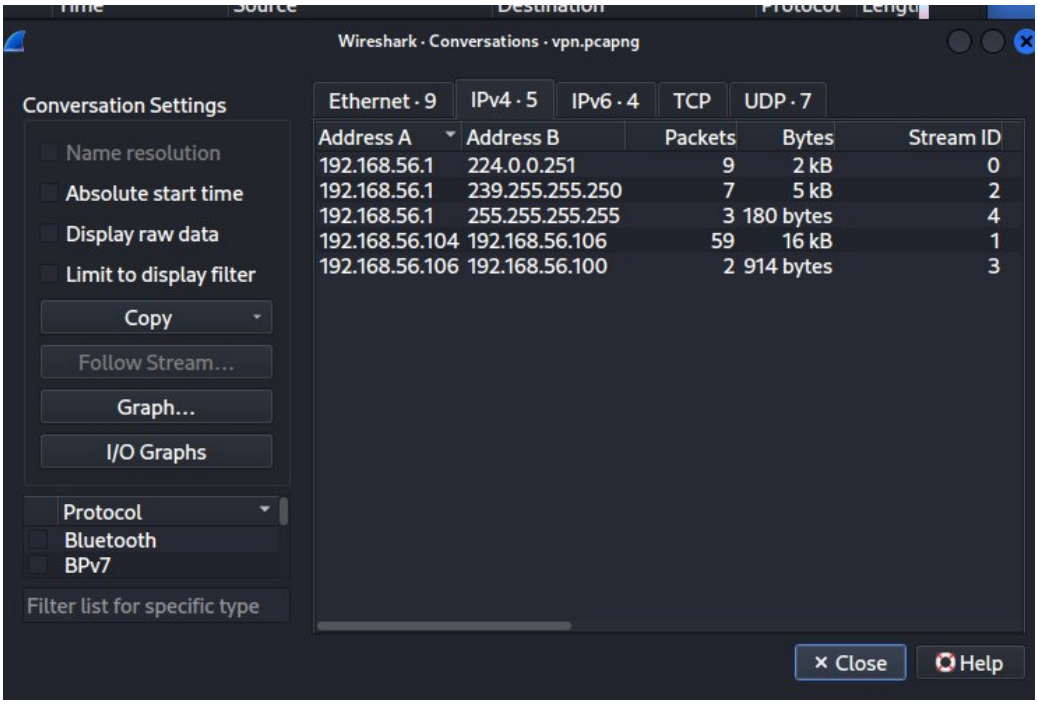


Figura 48: *Conversaciones IPv4 de la captura VPN.*

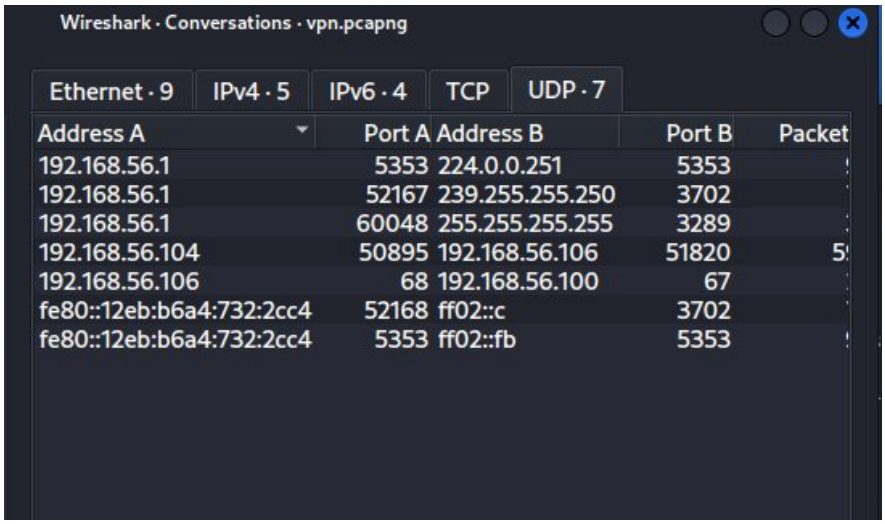


Figura 49: *Conversaciones UDP: el túnel usa 50895 → 51820.*

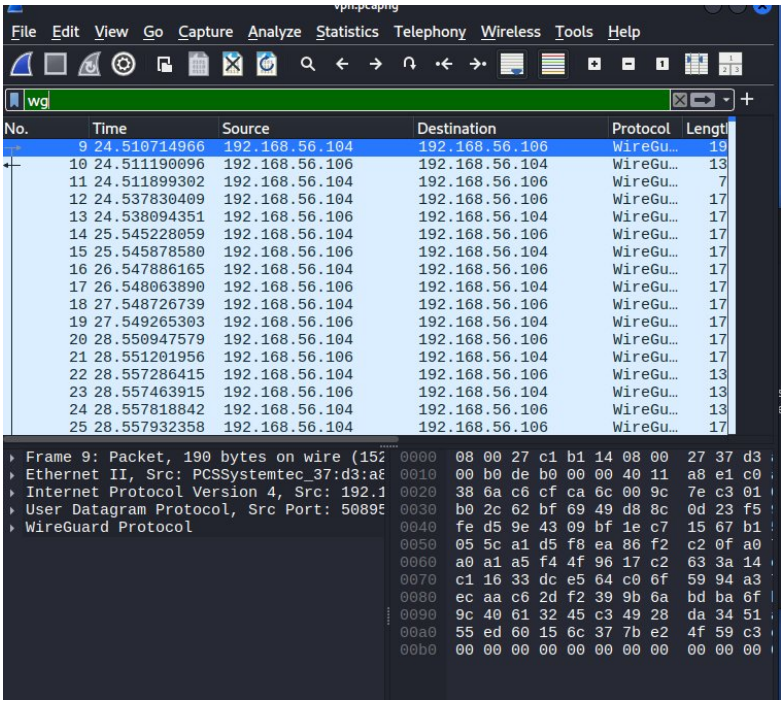


Figura 50: *Filtro wg: handshake y datos de transporte de WireGuard.*

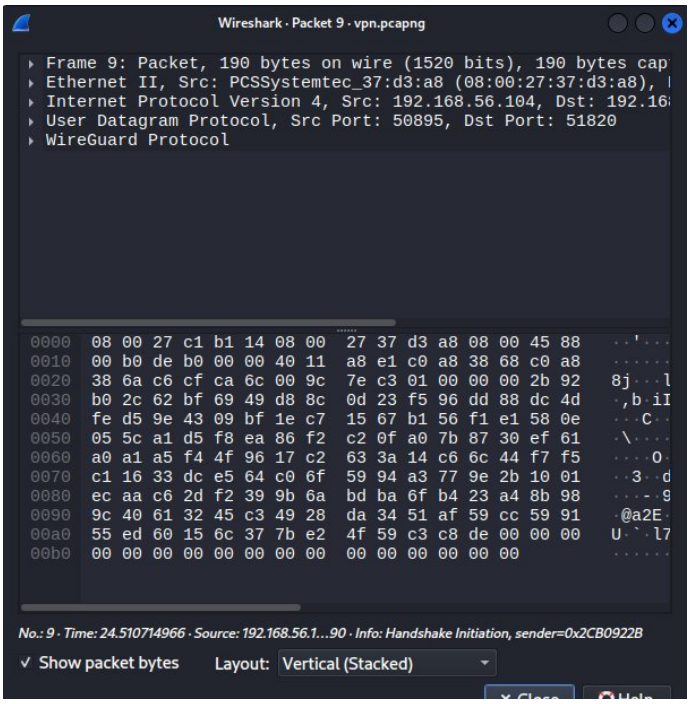


Figura 51: *Paquete 9: Handshake Initiation de 190 bytes.*

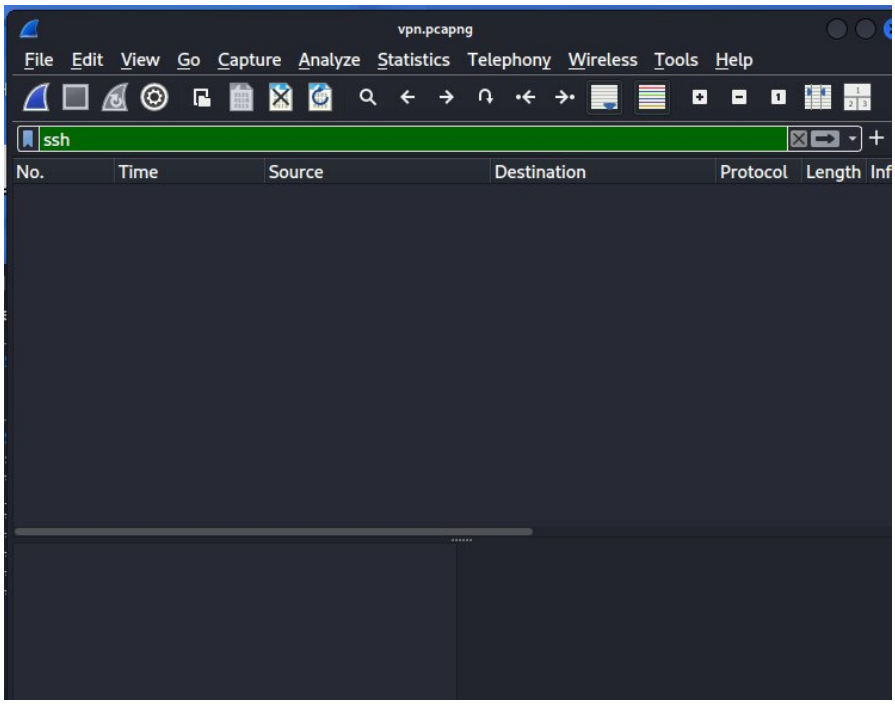


Figura 52: *Filtro `ssh` vacío en la captura VPN: la sesión SSH quedó oculta dentro del túnel.*

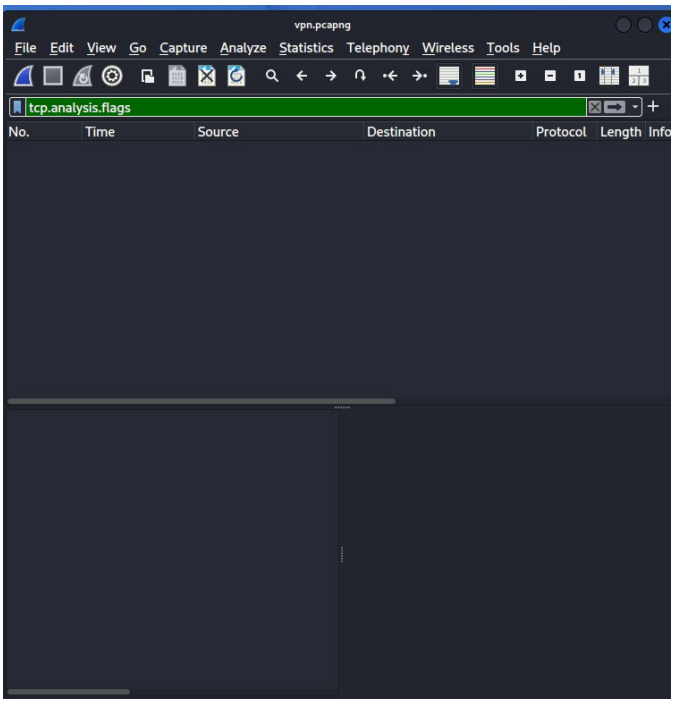


Figura 53: *Filtro `tcp.analysis.flags` vacío en la captura VPN.*

Tabla 9: *Resultados de referencia obtenidos con Wireshark.*

Aspecto	Captura SSH	Captura VPN (WireGuard)
Paquetes / duración	3.248 paquetes, 341 kB, 254 s	110 paquetes, 33 kB, 139 s
Protocolo principal	SSH sobre TCP (3.176 paquetes, 97,8 %)	WireGuard sobre UDP (59 paquetes, 53,6 %)
Otros protocolos	Solo TCP sin datos (saludo, ACK, cierre)	Tráfico de fondo: MDNS (18), XML/WS-Discovery (14), ARP (12), DHCP (2), ICMPv6 (2), datos UDP (3)
Cliente → servidor	192.168.56.104:37210 → 192.168.56.106:22	192.168.56.104:50895 → 192.168.56.106:51820
Otras IP	Ninguna	192.168.56.1 (PC anfitrión DESKTOP-5LMOD0P) y 192.168.56.100 (DHCP de VirtualBox)
Inicio de la conexión	Saludo TCP, versiones OpenSSH 10.2p1 (cliente) y 10.5p1 (servidor), *Key Exchange Init*, intercambio híbrido post-cuántico (mlkem768x25519-sha256) y *New Keys*	*Handshake Initiation* (paq. 9), *Handshake Response* (paq. 10), *Keepalive* (paq. 11) y luego *Transport Data*
Qué se ve	IP, puertos, versiones del software y algoritmos negociados	Solo IP, puertos y tipo de mensaje; desde el primer paquete el contenido está cifrado
Qué está cifrado	Usuario, contraseña, comandos y respuestas	Todo lo que va por el túnel, incluida la sesión SSH (el filtro `ssh` no muestra nada)
Anomalías	Ninguna (`tcp.analysis.flags` vacío)	Ninguna; el tráfico de fondo es normal de una red local

6.2. Rendimiento y consumo de recursos

Los datos salen de las estadísticas de `-verbose` de cada análisis. “Lectura” es la velocidad con la que el modelo procesa el prompt y “generación” la velocidad con la que escribe la respuesta.

Tabla 10: *Rendimiento en los seis análisis.*

Análisis	Modelo	Tiempo total	Tokens leídos	Lectura (tok/s)	Tokens generados	Generación (tok/s)
SSH	Llama 3.2 3B	20 min 25 s	2.590	17,98	584	0,55
SSH	Qwen 3 4B	1 h 21 min 35 s	4.004	9,07	2.371	0,53
VPN	Llama 3.2 3B	10 min 35 s	2.867	13,84	621	1,41
VPN	Qwen 3 4B	2 h 8 min 39 s	4.033	–*	3.361	0,44
Comparación	Llama 3.2 3B	15 min 41 s	5.243	13,18	456	0,84
Comparación	Qwen 3 4B	3 h 5 min 40 s	7.837	8,24	3.861	0,38
Total	Llama 3.2 3B	**46 min 41 s**			1.661	
Total	Qwen 3 4B	**6 h 35 min 54 s**			9.593	

*En el análisis VPN de Qwen, Ollama reutilizó el prompt que ya tenía en memoria (4.032 tokens), así que la velocidad de lectura no es comparable.

Se observa lo siguiente:

- **Tiempo de respuesta:** Qwen tardó en total unas **8,5 veces más** que Llama (6 h 36 min frente a 47 min). La mayor parte de lo que escribe Qwen es su razonamiento interno: aproximadamente entre el 70 % y el 90 % del texto generado fue *thinking* y solo el resto fue la respuesta final.
- **Velocidad:** Llama lee el prompt casi al doble de velocidad que Qwen (13–18 frente a 8–9 tokens/s). Al escribir, los dos bajaron a menos de 1,5 tokens/s, mucho menos que en la prueba corta del punto 4 (4,66 y 1,80 tokens/s). Esto pasa porque con prompts de 2.500 a 7.800 tokens cada palabra nueva tiene que "mirar" todo el contexto, y en una CPU sin GPU eso es muy costoso.
- **Recursos:** los dos usaron el 100 % de los 4 núcleos. Qwen ocupó cerca de 1 GB más de RAM (3,2–3,3 GB frente a 2,4–2,6 GB) y llevó la carga del sistema a 4,69 frente a 3,25. Ninguno necesitó swap.

6.3. Calidad de las respuestas frente a Wireshark

Cada respuesta se calificó contra la Tabla 9: **2** = correcta, **1** = parcial o imprecisa, **0** = incorrecta o inventada. En la comparación se usaron los cinco criterios que pedía el prompt.

Tabla 11: *Calificación de las respuestas de los análisis SSH y VPN.*

Pregunta	Llama SSH	Qwen SSH	Llama VPN	Qwen VPN
1. Protocolos	2	2	1	1
2. IP, cliente y servidor	2	2	1	2
3. Patrones de comunicación	1	2	1	2
4. Anomalías	2	2	2	2
5. Visible frente a cifrado	1	2	0	2
Total (máx. 10)	**8**	**10**	**5**	**9**

Tabla 12: *Calificación de la comparación SSH frente a VPN.*

Criterio	Llama 3.2 3B	Qwen 3 4B
Protocolos	2	2
Puertos	0	2
Patrones de comunicación	0	1
Cifrado	0	1
Visibilidad de la información	1	1
Total (máx. 10)	**3**	**7**

Puntaje global: Llama 3.2 3B obtuvo **16 de 30 (53 %)** y Qwen 3 4B **26 de 30 (87 %)**.

Errores y alucinaciones encontrados (información que no está en la captura o que es falsa):

- **Llama, comparación:** dice que WireGuard usa los puertos 1194 y 10000. En Wireshark se ve el 51820; el 1194 es el puerto típico de OpenVPN, no de WireGuard.
- **Llama, comparación:** afirma que la VPN usa más paquetes que SSH, cuando fueron 59 paquetes WireGuard frente a 3.248 de SSH. También dice que WireGuard usa "cifrado de bloque en cadena", pero WireGuard usa ChaCha20, que es un cifrado de flujo.
- **Llama, SSH:** dice que las claves de cifrado se ven en la captura. En Wireshark solo se ven los mensajes del intercambio; las claves nunca viajan en claro.
- **Llama, VPN:** inventa que 192.168.56.106 también es una dirección del cliente, y se contradice diciendo que WireGuard va en texto plano y cifrado al mismo tiempo.
- **Llama, prueba del punto 4 (Figura 8):** afirma que SSH lo desarrolló "Tivoli Systems". En realidad lo creó Tatu Ylönen en 1995.
- **Qwen, VPN:** dice que el protocolo principal es UDP. UDP es el transporte; el protocolo de la VPN es WireGuard (que sí menciona).

- **Qwen, comparación:** mete el tráfico MDNS, ICMPv6 e IPv6 como si fuera parte de la VPN, cuando en Wireshark se ve que es tráfico de fondo del PC anfitrión. También dice que el handshake de WireGuard es "más complejo", cuando solo tuvo 3 mensajes frente a unos 7 de SSH.
- **Qwen, SSH:** explica la pausa de 10,8 segundos entre el intercambio de claves y el primer dato cifrado como parte del proceso criptográfico. En realidad fue el tiempo que tardó el usuario en aceptar la huella del servidor y escribir la contraseña.

Lo que ninguno de los dos detectó: que en la captura VPN se hizo una sesión SSH completa por dentro del túnel y no aparece ni un solo paquete SSH (Figura 52). Los modelos no podían saberlo porque solo vieron el resumen de texto; esta conclusión solo sale de combinar Wireshark con el conocimiento de lo que se hizo en el laboratorio.

6.4. Síntesis de la comparación

Tabla 13: *Comparación final de los dos modelos.*

Criterio	Llama 3.2 3B	Qwen 3 4B	Mejor
Tamaño / parámetros	2,0 GB / 3B	2,5 GB / 4B	Llama
RAM durante la ejecución	$\approx$ 2,4–2,6 GB	$\approx$ 3,2–3,3 GB	Llama
Tiempo de los 3 análisis	47 min	6 h 36 min	Llama
Calidad (puntaje)	16/30 (53 %)	26/30 (87 %)	Qwen
Alucinaciones	Varias y graves (puertos, cifrado, claves visibles)	Pocas y menores (interpretación del tráfico de fondo)	Qwen
Uso de los datos de la captura	Respuestas genéricas, poco ligadas a los paquetes	Cita números de paquete, versiones y porcentajes	Qwen

En pocas palabras: **Llama es rápido pero poco confiable**, y **Qwen es mucho más preciso pero muy lento** en un equipo sin GPU. Para un análisis donde importa que los datos sean correctos, Qwen es claramente mejor; Llama sirve más para preguntas generales o explicaciones rápidas.

7. Conclusiones

Capacidades de la IA local

- Se pudo analizar tráfico de red con IA **sin que ningún dato saliera del equipo**, lo cual es lo más importante en ciberseguridad cuando se trabaja con información sensible de una red.
- Los dos modelos identificaron correctamente el protocolo principal, las direcciones IP y quién era cliente y quién servidor, y ninguno reportó falsas alarmas de ataques.
- Qwen 3 4B logró un análisis muy cercano al de Wireshark: reconoció el intercambio de claves, las versiones de OpenSSH, el puerto 51820 y diferenció el tráfico cifrado del que va en texto plano. Como apoyo para explicar y resumir una captura es útil.

Limitaciones

- **No leen capturas directamente:** hubo que convertir cada archivo a texto y resumirlo. Los modelos solo vieron 40 de los 3.248 paquetes de SSH, así que cualquier cosa fuera de ese resumen era invisible para ellos.
- **Son lentos sin GPU:** un análisis de Qwen llegó a tardar más de 3 horas. En un incidente real eso no es práctico.
- **Inventan información:** Llama dio puertos, algoritmos y datos históricos falsos con total seguridad. Si no se verifica con Wireshark, el analista podría sacar conclusiones equivocadas.
- **No tienen contexto:** ninguno descubrió que el SSH viajó dentro de la VPN, porque no sabían lo que se hizo en el laboratorio. Eso solo lo pudo concluir la persona con Wireshark.
- **Consumen todo el equipo:** mientras responden usan el 100 % del procesador y entre 2,4 y 3,3 GB de RAM.